SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1741**

CO1-AT2-Constraint-Based Problem Solving

Register Number	**Bandla Brahmendra Babu**
Name	**192312439**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS:3 **Total Marks:30**

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.
   The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

   Constraints:

   - The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:
a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Code of Conduct:

I _Brahmendra babu(192312439) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Brahmendra

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5
Application of Concepts	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6
Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5

QUESTION 1: BACKTRACKING SEARCH FOR DOCTOR SHIFT ASSIGNMENT

1.1 Problem statement

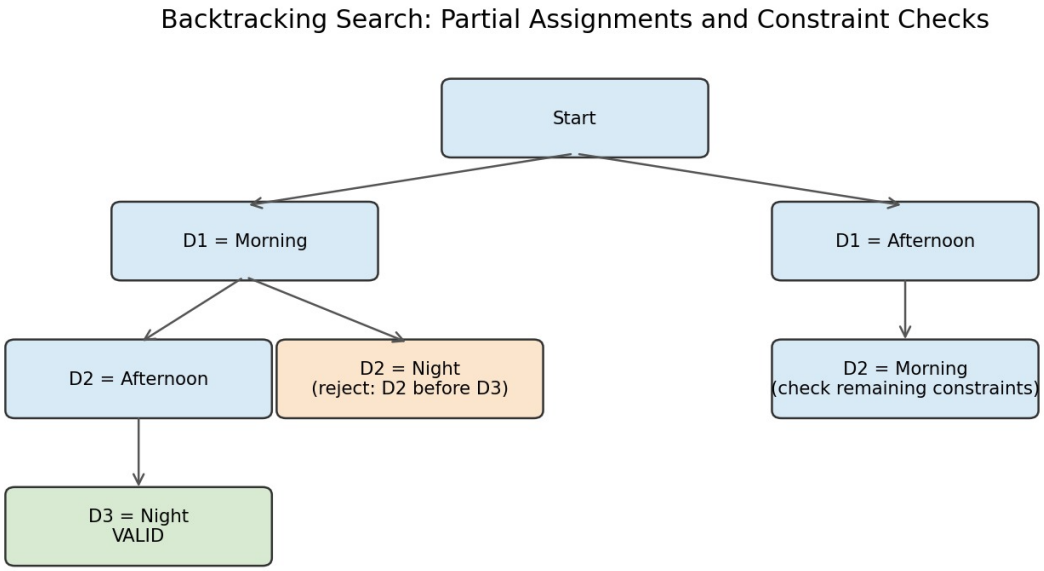
A hospital needs to assign three doctors—D1, D2, and D3—to three shifts: Morning, Afternoon, and Night. Each doctor must receive exactly one shift, and each shift must be occupied by exactly one doctor. The assignment must satisfy the ordering and availability constraints provided in the question.

1.2 Constraints

D1 cannot work the Night shift.
D2 must work before D3. For the three ordered shifts, this means D2's shift must occur earlier than D3's shift.
Only one doctor can be assigned to each shift.
D3 cannot work the Morning shift.
All doctors must be assigned exactly one shift.

1.3 What is backtracking search?

Backtracking is a systematic depth-first search technique for constraint-satisfaction problems. It builds a candidate solution one decision at a time. After every assignment, it checks whether the partial assignment is consistent with the constraints. If a constraint is violated, the algorithm rejects that branch, undoes the most recent assignment, and tries another value. This prevents the algorithm from spending time completing assignments that are already invalid.



1.4 Step-by-step search and constraint checks

Choose D1. The possible shifts are Morning and Afternoon because Night is prohibited for D1.
Select D1 = Morning. This assignment satisfies constraint (i).
Choose D2 from the remaining shifts, Afternoon and Night. Try D2 = Afternoon. The condition D2 before D3 requires D3 to be assigned to a later shift.
Assign D3 = Night. D3 is not in Morning, so constraint (iv) is satisfied. D2 is in Afternoon and D3 is in Night, so D2 works before D3.
Check uniqueness. D1, D2, and D3 occupy Morning, Afternoon, and Night respectively; no shift is shared.
Check completeness. Every doctor has exactly one shift. All constraints are satisfied, so the search returns this assignment.

1.5 Constraint-check table

Constraint	Check against final schedule	Status
D1 cannot work Night	D1 is assigned Morning	Satisfied
D2 before D3	Afternoon comes before Night	Satisfied
One doctor per shift	Each shift has one doctor	Satisfied
D3 cannot work Morning	D3 is assigned Night	Satisfied
Every doctor exactly one shift	D1, D2, D3 each appear once	Satisfied

1.6 Final valid schedule

Doctor	Assigned shift	Reason
D1	Morning	Not assigned to Night
D2	Afternoon	Works before D3
D3	Night	Not assigned to Morning; follows D2

1.7 Pseudocode

```
BACKTRACK(assignment):
if every doctor has a shift:
return assignment
choose an unassigned doctor D
for each shift s not already occupied:
assign D = s
if all constraints are satisfied:
result = BACKTRACK(assignment)
if result is successful:
return result
remove D = s
return failure
```

1.8 Why backtracking is appropriate

The problem is a constraint-satisfaction problem because it contains variables (doctors), domains (available shifts), and constraints (ordering, uniqueness, and exclusions). Backtracking checks constraints during construction rather than waiting until all possible schedules have been generated. For a larger hospital schedule, the algorithm could be improved by choosing the most constrained doctor first and checking constraints immediately after each assignment.

1.9 Conclusion

The final valid schedule is D1—Morning, D2—Afternoon, and D3—Night. The assignment satisfies all five constraints. Backtracking provides a clear and reliable way to explore possible schedules while rejecting invalid partial solutions.

QUESTION 2: BFS SEARCH FOR ROBOT NAVIGATION IN A 5×5 GRID

2.1 Problem statement

A robot must navigate from Start (S) to Goal (G) in a 5×5 grid. Some cells are blocked by obstacles. The robot may move Up, Down, Left, or Right, and each move has a cost of 1. The goal is to find an obstacle-free path with minimum cost.

2.2 Important note about the given data

The assessment statement does not specify the exact coordinates of S, G, or the obstacles. Therefore, the grid and path below are an illustrative example used to demonstrate the method. If the instructor provides a different obstacle map, the expansions and final path must be recalculated for that map.

2.3 Breadth-First Search (BFS)

BFS explores all nodes at the current distance from the start before exploring nodes at the next distance. It uses a first-in, first-out queue. When every move has equal cost, BFS finds a shortest path in number of moves. Each visited cell should be marked when it is first enqueued, and its parent should be recorded so the path can later be reconstructed.

Illustrative 5×5 Grid and One Shortest Path

S	·	·	#	·
•	·	#	·	·
•	#	·	·	·
•	•	·	#	·
·	•	•	•	G

2.4 Manhattan-distance heuristic

The Manhattan distance estimates the number of horizontal and vertical moves between a cell and the goal, ignoring obstacles:

h(n) = |row(n) − row(G)| + |column(n) − column(G)|

For example, from (0,0) to (4,4), the Manhattan distance is |0−4| + |0−4| = 8. Obstacles may force the robot to travel farther than this estimate.

BFS does not use the heuristic to order its queue. The assessment requests BFS and also mentions Manhattan distance; in this solution, the heuristic is shown as a distance estimate. A* search would use the heuristic to prioritize nodes, while BFS prioritizes nodes by discovery depth.

2.5 Step-by-step node expansion

Step	Node expanded	New valid neighbors / decision
1	(0,0)	Enqueue (0,1), (1,0)
2	(0,1)	Enqueue (0,2), (1,1)
3	(1,0)	Enqueue (2,0)
4	(0,2)	Enqueue (1,2); (0,3) is blocked
5	(1,1)	No new node; neighbors already visited or blocked
6	(2,0)	Enqueue (3,0)
7	(1,2)	Enqueue (2,2)
8	(3,0)	Enqueue (3,1), (4,0)
9	(2,2)	Enqueue (2,3)
10	(3,1)	Enqueue (4,1)
11	(2,3)	Enqueue reachable unvisited neighbors in queue order
12	(4,1)	Enqueue (4,2)
13	(2,4)	Enqueue (1,4), (3,4), if not visited
14	(4,2)	Enqueue (4,3)
15	(3,4)	Discover goal (4,4)

The exact expansion order depends on the chosen neighbor order (for example, Up, Down, Left, Right) and the illustrative obstacle map. The key BFS rule is that newly discovered cells are appended to the back of the queue and cells are removed from the front.

2.6 Priority queue / queue updates

BFS technically uses a FIFO queue, not a cost-priority queue. A sample queue progression is shown below. Entries are listed from front to back; the queue contains discovered cells that have not yet been expanded.

Operation	Queue after operation (illustrative)
Initialize	[(0,0)]
Expand (0,0)	[(0,1), (1,0)]
Expand (0,1)	[(1,0), (0,2), (1,1)]
Expand (1,0)	[(0,2), (1,1), (2,0)]
Expand (0,2)	[(1,1), (2,0), (1,2)]

2.7 Optimal path and cost

For the illustrative grid, one shortest path is:
(0,0) → (1,0) → (2,0) → (3,0) → (3,1) → (4,1) → (4,2) → (4,3) → (4,4)
Total cost = 8 moves × 1 per move = 8

2.8 BFS pseudocode

BFS(start, goal):
queue = [start]

```
visited = {start}
parent[start] = NONE

while queue is not empty:
current = remove_front(queue)
if current == goal:
return reconstruct_path(parent, goal)

for neighbor in [UP, DOWN, LEFT, RIGHT]:
if inside_grid(neighbor)
and not obstacle(neighbor)
and neighbor not in visited:
visited.add(neighbor)
parent[neighbor] = current
add_to_back(queue, neighbor)

return NO_PATH
```

2.9 Complexity and conclusion

For a grid with V reachable cells and E legal neighbor connections, BFS runs in O(V + E) time and uses O(V) memory. In a fixed 5×5 grid, the number of cells is small. BFS guarantees the minimum number of moves because all moves cost 1. Manhattan distance is useful for estimating remaining distance, but it does not change BFS's queue order.

QUESTION 3: AUTONOMOUS RESCUE ROBOT IN A PARTIALLY OBSERVABLE ENVIRONMENT

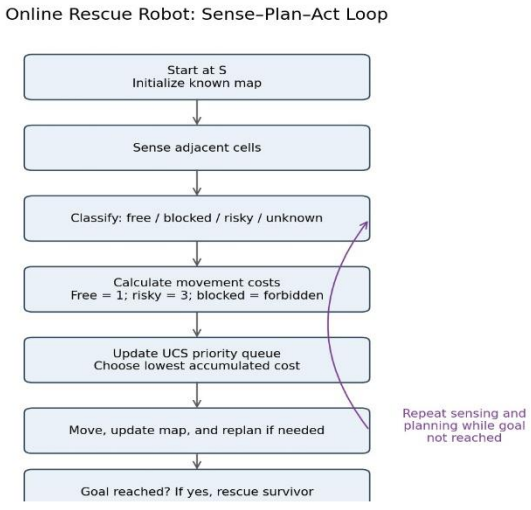
3.1 Problem overview

An autonomous rescue robot must travel through a disaster-affected building from a starting room S to a survivor at G. The building is represented as a grid. Some cells are blocked, the robot can observe only adjacent cells, and risky zones carry an additional cost. The robot must minimize travel cost while navigating safely and updating its knowledge online.

3.2 Analysis of constraints

Constraint	Effect on the robot's decision
Movement only Up, Down, Left, Right	The robot considers only orthogonal neighbors; diagonal moves are not allowed.
Blocked cells	The robot must never plan a path through known obstacles and must find alternatives.
Adjacent-cell sensing	The robot has incomplete knowledge and must explore to learn about unknown cells.
Normal move costs 1	Each ordinary transition adds one unit to the cumulative path cost.
Risky zone adds 2 extra	Entering a risky cell costs 1 + 2 = 3 units.
Minimize cost and navigate safely	The robot compares route costs, avoids obstacles, and applies safety rules to risky zones.
Dynamic knowledge / online search	The robot updates its map after sensing and replans when new information changes the route.

3.3 Proposed system block diagram



3.4 Selected approach: Uniform Cost Search with online exploration

Uniform Cost Search (UCS) expands the frontier state with the lowest accumulated path cost g(n). It is suitable when actions have nonnegative but unequal costs. The online agent complements UCS by sensing nearby cells, updating its internal map, and planning again as more of the building becomes known.

3.5 Step-by-step operation

Initialize at S. Set the known path cost g(S) = 0. Mark unobserved cells as unknown rather than assuming they are free.

Sense adjacent cells. Use the robot's sensors to classify nearby cells as free, blocked, or risky, according to the observations available.

Generate legal actions. Exclude blocked cells. For each known free neighbor, use cost 1; for each risky neighbor, use cost 3.

Calculate cumulative costs. For a transition from current cell u to neighbor v, calculate g(v) = g(u) + c(u,v).

Update the priority queue. Insert or update frontier nodes according to their lowest known cumulative cost. UCS removes the node with the smallest g-value.

Move to the selected node and observe again. Update the map with newly discovered cells and obstacles.

Replan if the new information affects the current route. The robot may discard an invalid route and run the cost-based search on the updated map.

Stop when the survivor location G is reached and the route is feasible under the safety constraints.

3.6 Example of cost comparison

Route	Normal moves	Risky moves	Total cost
A	6	1	6 + (1 × 3) = 9
B	8	0	8 + (0 × 3) = 8

Route B is preferable in this example because it costs 8 rather than 9 units and avoids risky zones. If a risky zone is prohibited by a strict safety rule, it must be treated as forbidden instead of merely assigning it a cost penalty.

3.7 Intelligent-agent concepts

A. Intelligent agents

An intelligent agent perceives the environment through sensors and acts through actuators to achieve a goal. In this problem, the sensors detect adjacent conditions, the agent program maintains a partial map and evaluates routes, and the robot's motors perform movement actions. Its goal is to reach the survivor safely at minimum cost.

B. Rationality

A rational agent selects the action expected to best achieve its performance measure using the information currently available. The robot avoids known obstacles, compares cumulative costs, accounts for risky cells, and changes its plan when it learns new facts. Rationality does not mean perfect knowledge: an online robot may initially choose a route that later proves unsuitable because unknown cells have not yet been observed.

C. Environment type

Property	Classification	Reason
Observability	Partially observable	Only adjacent cells can be sensed.
Determinism	Deterministic under the stated grid rules; real-world uncertainty may exist	Legal moves and their stated costs are defined, but physical conditions can add uncertainty.
Task structure	Sequential	Each movement affects future choices and route costs.
Dynamics	Potentially dynamic	New obstacles or hazards may be discovered or conditions may change.
State space	Discrete	The environment consists of grid cells.
Agents	Single-agent model	The problem specifies one robot as the decision-making agent.

3.8 Why this approach is suitable

Different movement costs: UCS correctly accounts for normal and risky transitions.
Cost optimality: with known transitions and nonnegative costs, UCS finds a minimum-cost path.
Obstacle avoidance: blocked cells are excluded from legal actions.
Adaptation: online sensing provides new information as the robot explores.
Replanning: the robot can revise its route when a newly observed obstacle or risk changes the best option.
Safety: if certain zones are unacceptable, hard safety constraints can prohibit entry rather than relying only on a penalty.
Limitation: UCS is optimal relative to the map and costs it is given. In a partially observable environment, the complete map is initially unavailable, so the online agent must repeatedly update and replan. A globally optimal route cannot always be guaranteed before unknown obstacles and risks are discovered.

3.9 Conclusion

Uniform Cost Search combined with online exploration is a suitable method for the rescue robot. The robot senses adjacent cells, avoids blocked areas, assigns a cost of 1 to normal moves and 3 to risky moves, and updates its map dynamically. It then chooses the least-cost feasible route using its current knowledge and replans when needed. This integrates intelligent-agent behavior, rational decision-making, and cost-aware navigation in a partially observable environment.